ORBITAL_X PROJECT

BACKEND ARCHITECTURE, CONTROLLERS, MODELS, SERVICES & API

BACKEND STACK:

- Runtime: Node.js (v18+)
- Web Framework: Express 5 (v5.2.1)
- Database ODM: Mongoose 9 (v9.9.5) with MongoDB / In-Memory MongoDB Server fallback
- Authentication: JSON Web Tokens (HMAC-SHA256) + Salted bcrypt (10 rounds)
- Security: Helmet (HTTP header hardening), CORS, compression
- Default Listening Port: 4000

TABLE OF CONTENTS

1. CORE SERVER & CONFIGURATION

1.1 server.js (Express 5 Entry Point, Telemetry Loop, In-Memory DB Fallback)

1.2 src/config/db.js (Resilient Dual-Mode Database Connection)

1.3 .env Configuration File

2. DATA MODELS (MONGOOSE SCHEMAS)

2.1 User.js (RBAC Roles: ADMIN, OPERATOR, ANALYST, VIEWER)

2.2 Satellite.js (Orbital Mechanics Parameters & Subsystem Health)

2.3 Mission.js (Lifecycle, Milestones & Satellite Associations)

2.4 GroundStation.js (Geographic Coordinates, Frequencies & Passes)

2.5 Telemetry.js (Sliding Window High-Frequency Historical Data)

2.6 Alert.js (Incident Tracking, Root Cause & Mitigation Status)

2.7 AuditLog.js (Immutable Security & Telecommand Audit Ledger)

2.8 Notification.js (Operator Priority Broadcast Messages)

2.9 Command.js (Executable Telecommand Queue & Flight History)

2.10 Communication.js (Encrypted RF Uplink/Downlink Ledger)

3. API CONTROLLERS

3.1 authController.js (Registration, Login & Profile Retrieval)

3.2 satelliteController.js (Fleet Inventory, Ephemeris & Telemetry)

3.3 telemetryController.js (Live Stream, Sliding Window & Aggregation)

3.4 alertController.js (Anomaly Lifecycle, Resolution & Injection)

3.5 aiController.js (Bridge to Python Port 5050 Diagnostic Service)

3.6 missionController.js (CRUD Operations & Satellite Allocations)

3.7 groundStationController.js (Network Monitoring & Radar Sweeps)

3.8 communicationController.js (Encrypted Command Dispatch & Terminal)

3.9 cubesatCommandController.js (Physical Subsystem Actuation)

3.10 adminController.js (System Parameters, Simulation Speed Multipliers)

3.11 userController.js (Account Management & Role Elevation)

3.12 auditController.js (Security Log Filtering & Export)

3.13 analyticsController.js (24h Fleet Aggregations & Reports)

4. MIDDLEWARE & SECURITY ENGINE

4.1 middleware/auth.js (JWT Verification & RBAC Authorization Guards)

4.2 middleware/errorHandler.js (Centralized Error Normalization)

5. BACKGROUND ENGINES & SERVICES

5.1 services/telemetrySimulator.js (Orbital Physics & Keplerian Propagation)

5.2 services/auditService.js (Automated Audit Record Creation)

6. UTILITIES, SEED DATA & TESTS

6.1 utils/seedData.js (Automatic Constellation & Admin Initializer)

6.2 utils/cubeSatInitialStates.js (Subsystem Baseline Metrics)

6.3 utils/password.js & token.js (Bcrypt & JWT Helpers)

6.4 utils/ApiError.js & ApiResponse.js (Standardized HTTP Envelopes)

6.5 tests/securityAudit.js (13-Stage Automated Security Test Suite)

CORE SERVER & CONFIGURATION

1.1 server.js (Express 5 Server Entry Point)

PURPOSE:

Initializes Express 5, establishes database connectivity, mounts global security middleware,

registers all 12 API route modules, seeds initial records, and starts the continuous telemetry loop.

DETAILED CODE ANALYSIS:

- Environment Setup: Invokes `dotenv.config()` to load `.env` parameters into `process.env`.

- Database Connection: Calls `connectDB()` from `./src/config/db`. If MongoDB is unavailable locally,

`connectDB()` automatically starts `mongodb-memory-server` so that the application operates out-of-the-box.

- Middleware Pipeline:

- `helmet()`: Hardens HTTP response headers (XSS filter, HSTS, frameguard).

- `cors({ origin: process.env.CLIENT_URL || 'http://localhost:4173', credentials: true })`:

Prevents cross-origin request forgery while allowing the Vite dev server access.

- `compression()`: Gzips response bodies over 1KB, optimizing bandwidth for large telemetry JSON streams.

  - `express.json()` and `express.urlencoded({ extended: true })`: Parses incoming request payloads.

  - `morgan('dev')`: Formats concise colored HTTP request logs in development.

- API Route Mounts:

  - `/api/auth` -> `authRoutes`

  - `/api/satellites` -> `satelliteRoutes`

  - `/api/telemetry` -> `telemetryRoutes`

  - `/api/missions` -> `missionRoutes`

  - `/api/ground-stations` -> `groundStationRoutes`

  - `/api/alerts` -> `alertRoutes`

  - `/api/communications` -> `communicationRoutes`

  - `/api/cubesat-commands` -> `cubesatCommandRoutes`

  - `/api/admin` -> `adminRoutes`

  - `/api/users` -> `userRoutes`

  - `/api/audit` -> `auditRoutes`

  - `/api/analytics` -> `analyticsRoutes`

  - `/api/ai` -> `aiRoutes`

  - `/api/notifications` -> `notificationRoutes`

- Telemetry Simulation Engine:

  - After database connection and seeding (`await seedData()`), executes:

      setInterval(simulateTelemetry, interval);

where `interval` defaults to 10,000 milliseconds (10s).

- Error Handling:

  - Unmatched routes fall through to a 404 handler returning `ApiError(404, 'API endpoint not found')`.

  - Global error handler `errorHandler.js` intercepts all uncaught exceptions, formatting standardized JSON.

1.2 src/config/db.js (Resilient Database Connection)

PURPOSE:

Provides self-healing database initialization. First attempts to connect to the MongoDB instance

specified in `MONGODB_URI` (default: `mongodb://127.0.0.1:27017/orbitalx`). If connection fails

(e.g., MongoDB service is stopped or not installed), it launches an in-memory MongoDB instance

using `MongoMemoryServer.create()`, connects Mongoose to its URI, and logs:

`[Database] In-Memory MongoDB Connected: 127.0.0.1`.

1.3 .env (Backend Environment Configuration)

PORT=4000

MONGODB_URI=mongodb://127.0.0.1:27017/orbitalx

JWT_SECRET=orbitalx_super_secret_jwt_key_2026_aerospace

CLIENT_URL=http://localhost:4173

SIMULATION_INTERVAL=10000

SEED_DUMMY_DATA=true

DATA MODELS (MONGOOSE SCHEMAS)

2.1 User.js

- Fields:

  - `name`: String, required.

  - `email`: String, required, unique, lowercase, trimmed.

  - `passwordHash`: String, required (bcrypt hash).

  - `role`: Enum `['ADMIN', 'OPERATOR', 'ANALYST', 'VIEWER']`, default `'OPERATOR'`.

  - `department`: String (e.g. 'Flight Dynamics', 'Payload Ops').

  - `lastLogin`: Date.

  - `isActive`: Boolean, default true.

- Role-Based Access Control (RBAC):

  - `ADMIN`: Full control; user role elevation, rate changes, audit export.

  - `OPERATOR`: Telecommand dispatch, alert acknowledge/resolve, simulation control.

  - `ANALYST`: View-only telemetry, analytics export, report generation.

  - `VIEWER`: Read-only access to dashboards and 3D globe.

2.2 Satellite.js

- Fields:

  - `satelliteId`: Unique String (e.g., 'SAT-ALPHA-1', 'CUBESAT-ION').

  - `name`: Display name.

  - `type`: Enum `['COMMUNICATION', 'OBSERVATION', 'WEATHER', 'CUBESAT_1U', 'CUBESAT_3U', 'CUBESAT_6U', 'CUBESAT_12U']`.

  - `formFactor`: Enum `['1U', '2U', '3U', '6U', '12U', '16U']`.

  - `orbitType`: Enum `['LEO', 'MEO', 'GEO', 'SSO', 'HEO']`.

  - `status`: Enum `['ACTIVE', 'STANDBY', 'DEGRADED', 'DECOMMISSIONED']`.

  - `inclination`: Number (Orbital tilt in degrees, e.g. 51.6 for ISS orbit).

  - `orbitRadius`: Number (Kilometers from Earth center, e.g. 6871 km for 500 km LEO).

  - `orbitSpeed`: Number (Velocity in km/s, e.g. 7.6 km/s).

  - `subsystems`: Nested object tracking status (`NOMINAL`, `WARNING`, `CRITICAL`) for:

    - `eps`: Electrical Power Subsystem

    - `adcs`: Attitude Determination & Control

    - `comms`: RF Telecommunications

    - `tcs`: Thermal Control Subsystem

- `obc`: On-Board Computer

- `payload`: Scientific/Commercial Sensors

2.3 Mission.js

- Tracks aerospace operational missions (e.g. 'MSN-APOLLO', 'Earth Watch 2026').

- Fields: `missionId`, `name`, `objective`, `status` (`PLANNING`, `ACTIVE`, `COMPLETED`, `ABORTED`),

`launchDate`, `launchSite`, `assignedSatellites` (Array of ObjectId references to `Satellite`),

`budget`, `flightDirector`.

2.4 GroundStation.js

- Represents ground tracking antenna sites across the globe.

- Fields: `stationId` ('GS-001'), `name` ('Bangalore Deep Space Network'), `country`,

`latitude` (e.g., 12.9716), `longitude` (e.g., 77.5946), `elevationMeters`,

`frequencyBands` (Array: `['S-BAND', 'X-BAND', 'UHF']`), `dishDiameterMeters`,

`status` (`ACTIVE`, `MAINTENANCE`, `OFFLINE`).

2.5 Telemetry.js (Sliding-Window Historical Metrics)

- High-frequency time-series record captured for every satellite.

- Fields: `satelliteId`, `timestamp`,

  - EPS: `batteryLevel` (%), `solarArrayOutput` (W), `voltage` (V), `current` (A), `batteryTemp` (K)

  - ADCS: `pitch`, `roll`, `yaw` (deg), `gyroRate` (deg/s), `reactionWheelRpm`

  - COMMS: `uplinkSignalDbm`, `downlinkSignalDbm`, `snrDb`, `vswr`

  - TCS: `cpuTemp` (K), `batteryTemp` (K), `externalTemp` (K)

  - OBC: `cpuLoad` (%), `memoryEccErrors`, `i2cBusAckPct`

  - Mechanics: `altitudeKm`, `velocityKmS`, `latitude`, `longitude`

- Retention: Maintained with a sliding window capped at 300 samples per satellite to avoid memory bloat.

2.6 Alert.js

- Incident records triggered by threshold breaches or fault injection.

- Fields: `satelliteId`, `type`, `severity` (`CRITICAL`, `WARNING`, `INFO`),

  `metric`, `value`, `threshold`, `message`, `status` (`ACTIVE`, `ACKNOWLEDGED`, `RESOLVED`),

`acknowledgedBy`, `resolvedBy`, `aiDiagnosis` (Object containing root-cause analysis and mitigation command).

2.7 AuditLog.js

- Append-only security ledger.

- Fields: `userId`, `action` (`USER_LOGIN`, `COMMAND_DISPATCH`, `ALERT_RESOLVED`, `ROLE_MODIFIED`),

`resource`, `ipAddress`, `status` (`SUCCESS`, `FAILURE`), `details`, `timestamp`.

API CONTROLLERS

3.1 authController.js

- `register`: Validates user inputs, checks for duplicate email, hashes password with bcrypt (10 rounds),

creates User record, generates JWT, and logs audit record.

- `login`: Sanitizes inputs (defends against NoSQL injection), checks user existence, compares bcrypt hash,

emits cryptographic JWT signed with HMAC-SHA256, returns user profile and token.

- `getMe`: Returns currently authenticated user session.

3.2 satelliteController.js

- `getAllSatellites`: Returns full fleet with latest telemetry summary and subsystem statuses.

- `getSatelliteById`: Detailed view with past 50 telemetry points and assigned mission.

- `createSatellite`: Operator endpoint to register a new satellite or CubeSat.

- `updateSatellite`: Updates physical/orbital parameters.

- `deleteSatellite`: Decommissions a satellite asset.

3.3 telemetryController.js

- `getLatestTelemetry`: Returns newest telemetry state across all active satellites.

- `getHistoricalTelemetry`: Retrieves sliding-window historical series filtered by `satelliteId` and time range.

- `getFleetStats`: Returns global fleet health percentages, average battery, link availability.

3.4 alertController.js

- `getAlerts`: Returns alerts filtered by status (`ACTIVE`, `ACKNOWLEDGED`, `RESOLVED`, `ALL`).

- `createAlert`: Creates an alert (used by the Anomaly Injector Drawer and automated threshold watcher).

- `acknowledgeAlert`: Marks alert acknowledged by operator.

- `resolveAlert`: Resolves alert, stores resolution timestamp, restores satellite subsystem status.

- `getAlertStats`: Aggregates active critical, warning, and 24h resolved totals.

3.5 aiController.js

- `diagnoseTelemetry`: Acts as an internal gateway to the Python AI Microservice (`http://127.0.0.1:5050/diagnose`).

  Forwards incoming telemetry vector, retrieves root-cause analysis, SOP checklist, and recommended telecommand.

  If the Python microservice is offline, gracefully falls back to deterministic heuristic aerospace rules.

4. MIDDLEWARE & SECURITY ENGINE

4.1 middleware/auth.js (JWT & RBAC Guards)

- `protect`:

  - Inspects `req.headers.authorization` for `Bearer <token>`.

  - Verifies cryptographic signature using `jwt.verify(token, process.env.JWT_SECRET)`.

  - Retrieves user from database (omitting password hash) and attaches to `req.user`.

  - If token is missing, expired, or forged, immediately halts execution with HTTP 401.

- `authorize(...roles)`:

- Higher-order middleware checking if `roles.includes(req.user.role)`.

- If user lacks permission (e.g. `VIEWER` attempting telecommand dispatch), returns HTTP 403 Forbidden.

4.2 middleware/errorHandler.js

- Normalizes errors across Mongoose CastErrors (invalid ObjectIds), validation errors, and custom `ApiError`.

- Formats standard JSON:

```
{
  "success": false,
  "status": 404,
  "message": "Resource not found",
  "stack": "..." (only in development)
}
```

BACKGROUND ENGINES & SERVICES

5.1 services/telemetrySimulator.js (Orbital Physics Engine)

- Runs continuously on a 10-second timer.

- For each active satellite:

1. Computes delta time and calculates updated True Anomaly / Mean Anomaly along its Keplerian orbit.

2. Updates latitude and longitude:

lat = asin(sin(inclination) * sin(meanAnomaly)) * (180 / PI)

lng = ((nodeLng + atan2(cos(inclination) * sin(meanAnomaly), cos(meanAnomaly)) - earthRotation) % (2 * PI))

3. Modulates solar array power based on whether satellite is in orbital sunlight vs. Earth eclipse shadow.

4. Simulates battery charge/discharge cycle.

5. Evaluates threshold alerts: If battery temp > 315K or VSWR > 3.0, automatically spawns an `Alert` record.

6. Stores telemetry in `Telemetry` collection and prunes excess records older than the sliding-window threshold.

UTILITIES, SEED DATA & TESTS

6.1 utils/seedData.js

- Automatically runs on cold boots:

- If database contains 0 users, seeds default Admin: `admin@orbitalx.com` / `admin123`.

- If `SEED_DUMMY_DATA=true`, seeds:

- 8 Satellites across LEO, MEO, GEO, SSO, and 1U-16U CubeSats.

- Apollo Operations Mission.

- 5 Global Ground Stations (Houston, Canberra, Kiruna, Bangalore, Toulouse).

- Initial Audit Logs.

6.2 tests/securityAudit.js (13-Stage Automated Security Suite)

- Validates the 5 security pillars:

1. Route Protection: Verifies unauthenticated access to `/api/telemetry`, `/api/missions`, `/api/satellites`, `/api/users` is blocked (HTTP 401).

2. Cryptographic Integrity: Rejects forged JWT tokens and empty Bearer strings.

3. NoSQL Injection Defense: Verifies `{"$gt": ""}` operator injection in username/password fields is neutralized.

4. Role Authorization: Verifies `ADMIN` privilege elevation and restriction enforcement.

5. Payload Sanitization: Asserts 404 behavior for invalid object IDs and malformed routes.